

HTTP Protocol Behavior Lab: Exploring HTTP/1.1

1. Objective

This lab experimentally analyzed key HTTP/1.1 application-layer concepts using real network requests and timing measurements. The study focused on HTTP status codes, persistent connections (keep-alive), and head-of-line blocking behavior.

2. Tools Used

- Python 3
- requests, pandas, matplotlib
- curl
- Test endpoints: <https://example.com> and <https://httpbin.org>

3. Procedure

Experiment 1: Status Codes

Used curl and Python requests to query different endpoints returning 200, 301, 404, and 500 responses. Results were categorized and plotted.

Experiment 2: Keep-Alive vs No Session

Ten repeated HTTP GET requests were sent without a session (new connection behavior) and ten requests were sent using `requests.Session()` to reuse connections. Latency was measured for every request.

Experiment 3: Head-of-Line Blocking

Five delayed requests were sent sequentially to <https://httpbin.org/delay/2> and then five delayed requests were sent in parallel using `ThreadPoolExecutor`. Total completion time was compared.

4. Measured Results

No Session Mean Latency: 0.2139 s

Session Keep-Alive Mean Latency: 0.0428 s

Latency Reduction: ~80%

Sequential Delayed Requests: 12.4858 s

Parallel Delayed Requests: 2.3579 s

Parallel Speedup: 5.30x

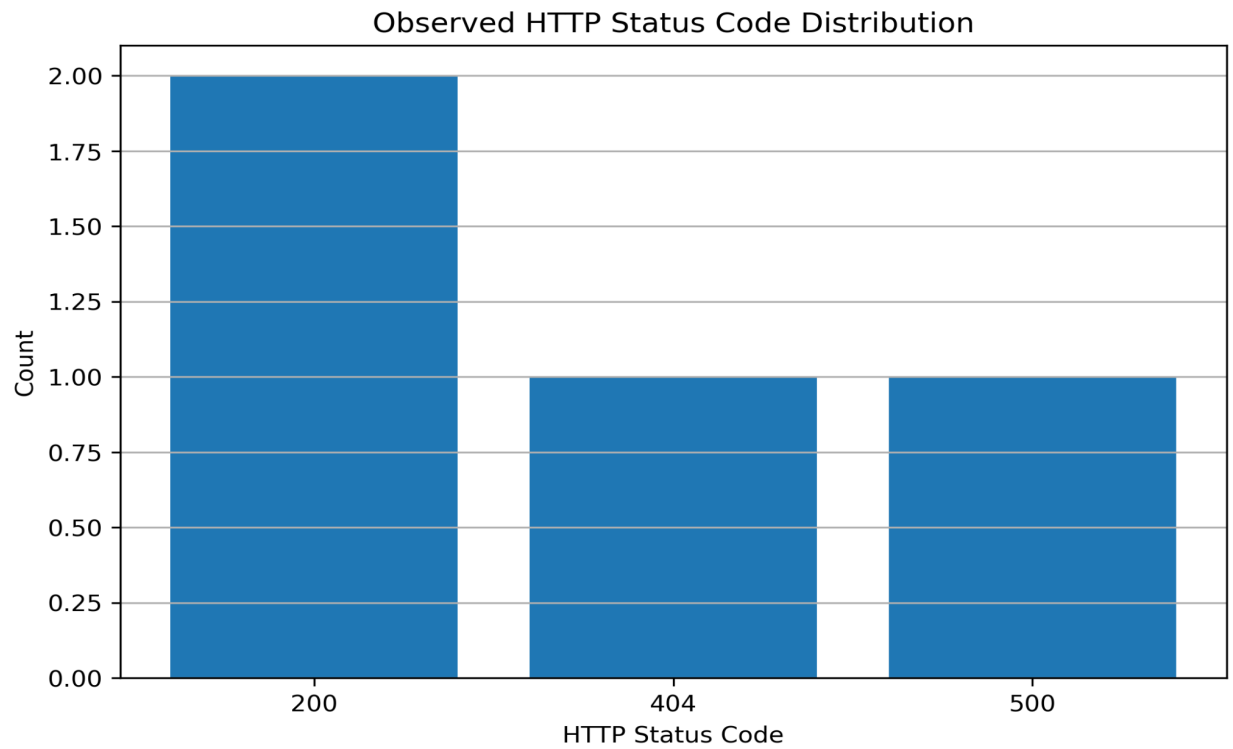
5. Interpretation

Persistent connections significantly reduce repeated request overhead by avoiding repeated TCP/TLS setup costs. Sequential delayed requests demonstrate head-of-line blocking because each request waits for previous responses before completion. Parallel requests mitigate this effect.

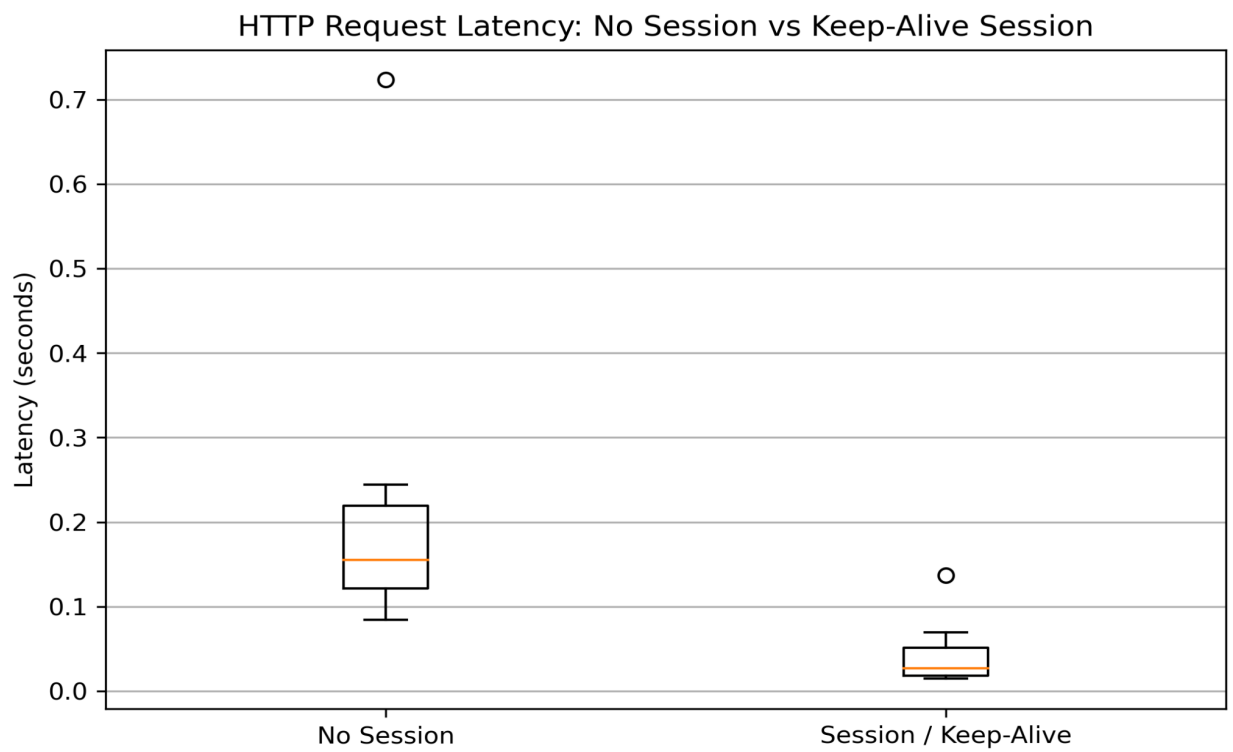
6. Conclusion

The experiments show that HTTP/1.1 design choices directly influence performance. Keep-alive improves efficiency, while sequential response ordering can reduce responsiveness under delay.

Observed HTTP Status Code Distribution



HTTP Request Latency: No Session vs Keep-Alive



Head-of-Line Blocking: Sequential vs Parallel Requests

Head-of-Line Blocking: Sequential vs Parallel Requests

